

Stage 2 — Test Automation Evidence

Canteen Ordering System

Test Suite Overview

- Framework: JUnit 5 + Mockito (unit tests), Spring Boot Test / MockMvc (integration tests)
- Location: backend/src/test/java/com/canteen/
- unit/OrderServiceTest.java — 18 tests, business rules tested in isolation
- integration/OrderControllerIT.java — 7 tests (updated to 8 with the concurrency test) against a real, in-memory, test-only H2 database via the full HTTP stack
- Total: 23 tests, 23 passing on a clean run (2 sec 933 ms)

Coverage Matrix

Rule	Normal path	Edge case	Invalid input
Cutoff time	Order before cutoff succeeds	Exactly at cutoff / 1s before / 1s after	—
Stock	Order within stock succeeds	Exactly last unit succeeds; exceeding stock rejected	—
Daily limit	Order under limit succeeds	Exactly at limit succeeds; exceeding limit rejected	—
Cancellation	Cancel within grace window succeeds, restores stock	Cancel at window boundary	Non-owner cancel rejected; double-cancel rejected
Quantity	—	—	Zero / negative quantity rejected
Auth	Valid token succeeds	—	Missing token → 401; tampered price field ignored
Concurrency	—	Two simultaneous requests for the last unit: exactly one succeeds, one gets 409	—

Deliberate Red Run

What we broke

The stock-availability check in `OrderService.placeOrder()` was changed from a correct less-than comparison to an equality comparison — a subtle, realistic logic error (the kind that passes a quick code review):

```
// Before (correct):
if (item.getStockCount() < quantity) {
    throw OrderException.outOfStock(item.getName());
}

// After (broken):
if (item.getStockCount() == quantity) {
    throw OrderException.outOfStock(item.getName());
}
```

Result

4 tests failed, 19 passed (23 total, 3 sec 18 ms).

Why this breaks things

The `==` comparison only rejects an order when the requested quantity exactly matches the remaining stock — every other case behaves wrong in one of two directions. Ordering *more* than the available stock no longer gets rejected at all (since stock count will never exactly equal an over-large quantity), letting stock go negative. Ordering *exactly* the last unit — a case that should succeed — gets wrongly rejected instead, since that is precisely the case the equality check now blocks. This inverted logic also affects the concurrency test: because the stock guard no longer reliably blocks an over-large request, the two-thread race test that expects exactly one of two simultaneous orders for the last unit to succeed instead sees unexpected behaviour, failing alongside the direct stock-boundary tests.

Fix

Reverted the comparison back to `<` and re-ran the full suite.

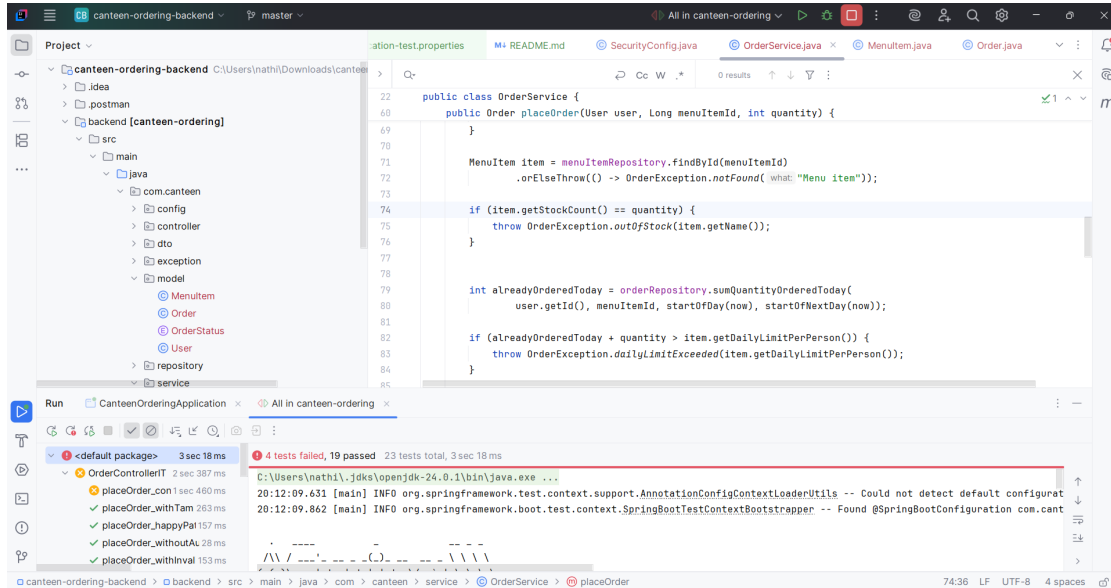
Result after fix: 23 tests passed, 23 total, 2 sec 881 ms.

What This Demonstrates

- The suite is not “always green” by construction — it fails when the code it is testing is actually broken, including subtle off-by-one-style logic errors, not just when code is deleted outright.
- Multiple, independent tests failed for logically connected reasons (stock-boundary handling and the concurrency guard both depend on the same comparison), which is itself useful diagnostic signal — a cluster of related failures pointed straight at the one line that changed.
- The concurrency test (`placeOrder_concurrentRequestsForLastUnit_onlyOneSucceeds`) is a genuine race-condition test using two threads and a `CountDownLatch` against a real (in-memory) database — not mocked — so it actually exercises JPA's optimistic locking behavior end to end, and is sensitive enough to fail when the stock guard it depends on is wrong.

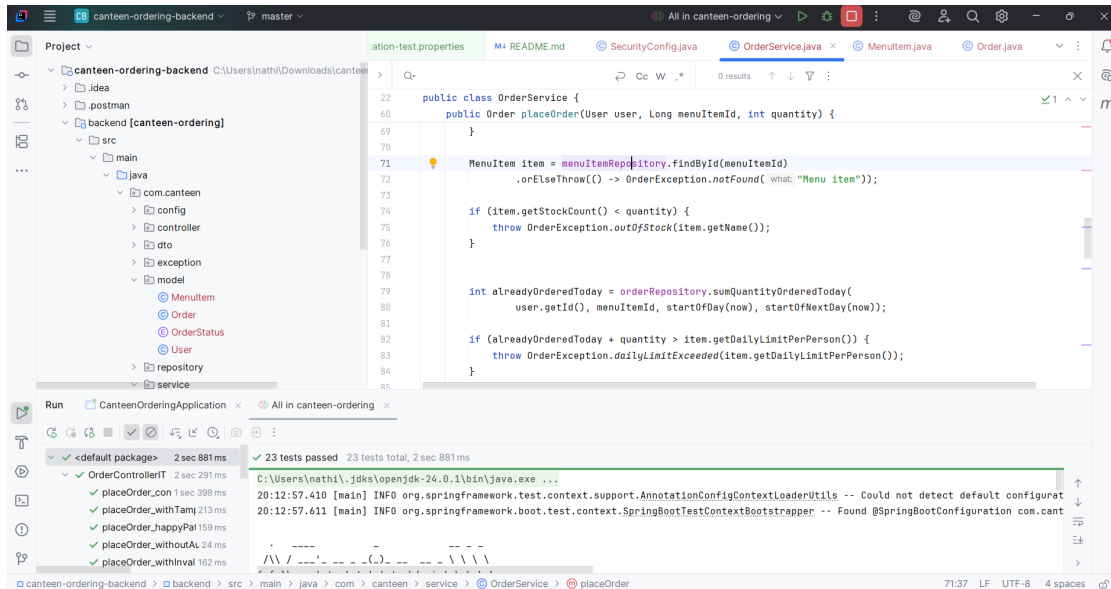
Supporting Screenshots

Red run — stock check changed to == quantity, 4 tests failing:



4 tests failed, 19 passed, 23 total, 3 sec 18 ms.

Green run — stock check reverted to < quantity, all tests passing:



23 tests passed, 23 total, 2 sec 881 ms.